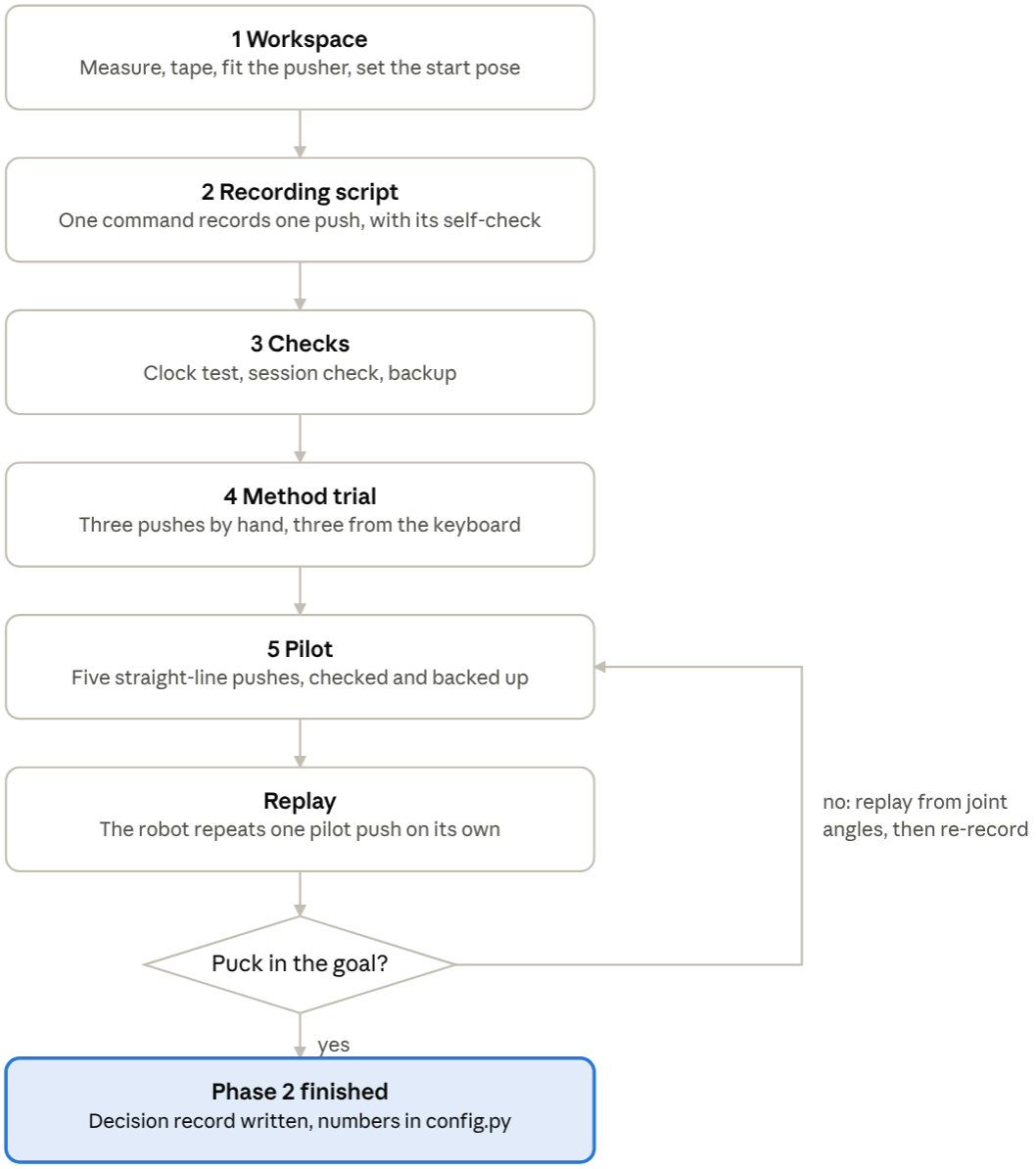
Phase 2 Guide: Recording

Oct 24, 2026 · @Emranul Hoque

How Team M records its first demonstrations: the workspace, the recording script, the checks, the choice between guiding by hand and driving from the keyboard, and the five-episode pilot that ends the phase. Only this phase; training and everything after it get their own guide when we reach them. Every unfamiliar word is explained where it first appears, and the big picture is in the AI Project Guide.

This phase at a glance

Five steps and one replay take us from an empty table to a checked pilot



phase 2 · 5 steps, 1 replay, 1 decision

Each step feeds the next. A replay that misses the goal sends us back to the pilot with a different kind of recording, not back to the start.

What you do by hand, and what the scripts do

Most of this phase is hands-on, and most of the hands-on time is not pushing. It is taping, measuring, placing the puck and looking at pictures. The scripts are written once, with me, and from then on they do the recording, the checking and the copying every time.

Part of the phase	By hand	By script	Time, roughly
Workspace (step 1)	Measure the puck and the line, tape the sheet, fit the pusher, place the puck, take photos	Move the arm to the start pose and store it	2 to 3 hours, once
Recording script (step 2)	Press start, stop and discard; type the outcome; watch the arm	Everything else: pictures, joint angles, tip position, time stamps, file names, folders, the integrity check	Half a day of writing with me, then nothing
Checking (step 3)	Tap the pusher on the table three times for the clock test; look at the plots	The clock test, the file checks, the plots, the contact sheet, the backup copy	10 minutes per session
Method trial (step 4)	Three pushes by hand and three from the keyboard	Score the pushes	30 to 45 minutes
Pilot (step 5)	Five pushes, a reset between them, one replay watched	Record, check, back up, replay	30 to 45 minutes

One push, from "puck at the start" to "script stopped", takes about a minute by hand and two to three minutes from the keyboard, plus half a minute to put the puck back. That is the number to remember: every recording session we ever do is a multiple of it.

Step 1: the workspace

Everything the robot sees and touches is fixed in this step, and then never moved again without a note. The recordings only help if the exam set-up looks like the recording set-up.

1. Measure the puck with a ruler and a kitchen scale: diameter, height and weight. Write them into `config.py`. The diameter sets how close "in the goal" is: the puck's centre within one puck radius, half the diameter, of the end of the line. The height sets the pushing height: the pusher touches the puck below half its height, so the push cannot tip it over. The weight is only for the record; a wooden puck is far too light to trouble the robot.

2. Tape the straight-line sheet flat on the table in front of the robot, all four corners, and mark the corners on the table with small pieces of tape so the sheet can go back to the same place. Measure the line's length with a tape measure and write it down.
3. Fit the pusher. The gripper is off, and the pusher is the bare steel shaft that is left at the arm's wrist; nothing is attached to it, because a round contact can steer the puck around bends and a flat face cannot. Check by hand that it touches the puck below half the puck's height and that the camera still sees the puck and the line ahead of it. Measure the shaft's diameter and how far it reaches below the wrist, and write both into config.py. If the professor names a different pusher on 26.10.26, repeat points 3, 4, 5 and 7 with it before the method trial.
4. Find the pushing height: hold the arm at the elbow, then switch learning mode on (the motors go limp so the arm can be moved by hand; it no longer holds its own weight, so keep holding the elbow the whole time it is on). Bring the pusher down until it touches the puck's side at the right height, run `check_hardware.py`, which prints the tip position, and write the height into `config.py`.
5. Set the start pose: the pusher at pushing height, directly behind the puck at the start of the line, on the side away from where the line goes. Put the tip there the same way as in point 4 and read x, y and z from `check_hardware.py`. A short script moves the arm there from the home pose and back; we store the pose in `config.py` and run the same script before every recording.
6. Define the workspace box: put the tip on each of the four sheet corners the same way and read x and y, widen the box by 5 cm on every side, and write it into `config.py`. Any step the arm is later asked to make outside this box is refused.
7. Photograph the set-up with a tape measure in the frame: the sheet, the robot base, the pusher at the start pose, the lighting. The photos go into the documentation folder, so the set-up can be rebuilt exactly for every session and for the exam.

Manual work: the whole step is by hand, two to three hours once. After that only the taping and the start script repeat, a few minutes per session.

Step 2: the recording script

One command records one push. I write the script with you, in our code style, and you run it. This is what it does.

What it saves, into one folder per session, for every push:

What	File	Why
The camera pictures, at the camera's own rate	`path_01_straight_line_episode_001.mp4`, a video	What the robot will see
One row per picture, about nine per second (the camera's real rate, measured on 24.10.26), written as each picture arrives so that row n belongs to picture n: time, six joint angles, tip position (x, y, z and tilt), learning mode on or off	`path_01_straight_line_episode_001.csv`, a spreadsheet-style text file	Where the arm was; the labels come from here later
Which sheet, who pushed, by hand or keyboard, and whether the push succeeded	One line added to `session_notes.md`	So a bad push can be skipped later without deleting anything

The folder is `03_data/raw/2026-10-27_session_01/`, named by date and session number. File names start with the sheet's name and an episode number that counts up for that sheet across all sessions, so `path_01_straight_line_episode_004` exists exactly once. Nothing in this folder is ever edited, renamed or deleted; a bad push is noted and skipped.

Only one of our scripts may be connected to the robot at a time. A second one hangs at connection and never answers, while the first keeps working (measured on 24.10.26). NiryoStudio may stay open; it uses other channels.

The keys you press:

- Type the sheet's name once at the start of a session.
- `r` starts a push, `s` stops it, `d` throws the push just made away before it is saved.
- After each push the script asks for the outcome: success, fail, or a note.

After each push the script checks itself: the number of pictures matches the video's length, no gap in the table longer than two ticks, no empty file. A push that fails this check is marked, and you simply redo it.

Manual work: three keys and one word per push; the rest is the script. Writing and testing the script together at the robot: about half a day.

Step 3: checking a recording

Two checks tell us a recording is usable before we trust it with hours of work.

The clock test, once per session, before the first push. The pictures and the robot's state are two separate streams, each with its own time stamps. If they are out of step by even a tenth of a second, the policy learns a move that belongs to a picture from a moment earlier. So at the start of a session, with learning mode on and one of you holding the elbow, lift the tip about 2 cm and tap it on the table three times, about a second apart, where the forearm camera can see it. The script finds each tap in the pictures and in the table and writes the offset between the two clocks into `sync.json`. If the three taps disagree by more than one picture interval, about 110 ms, the session fails and we look for the cause before recording anything.

The session check, after the last push. One script reads the whole session folder and reports:

- every push has both its video and its table, and the episode numbers run without a gap;
- the self-check of each push passed;
- the line is visible in every picture: in a shrunk picture at least 0.5 % of the pixels are dark line pixels (a number we tune on the pilot);
- a drawing of the tip's path over a photo of the sheet, one per push, so you see at a glance whether the puck followed the line;
- a contact sheet, one picture out of every 30 from each push, side by side, so a hand in view or a blurred stretch is spotted in seconds;
- the session is copied to the second disk (the external drive kept for backups, its path in `config.py`) and the copy is verified.

Reading the output: a session that passed says so in one line; anything that failed is named with its reason. A failed push stays in the folder, is written into the exclusion list (a text file in `03_data` naming every episode that must never be used), and is never used.

Manual work: three taps and ten minutes of looking at plots per session.

Step 4: hand-guiding or keyboard driving

We try both ways on the straight line, three pushes each, after the professor has answered question 4 at the meeting on 26.10.26, or with our own default if he leaves it to us.

Hand-guiding: learning mode switches the motors off; one of us holds the arm at the elbow and pushes, the other runs the script. Keyboard driving: the motors stay on; one of us presses keys that move the tip a few millimetres each, and the other watches the puck.

What we measure on each push	Why it matters	Likely winner
Smoothness of the tip's path	A jerky path gives steps that zigzag, and the policy copies them	Keyboard
Puck within one puck radius of the line in every picture	The policy copies what it sees	Whichever was steadier
Height and tilt stayed within 5 mm and 5 degrees	When the robot drives itself, height and tilt are fixed	Keyboard
No hand in any picture	The policy would learn to look for hands	Keyboard
Time per push	Every later session is a multiple of it	Hand

The six trial pushes stay in the session folder under the next six free numbers (straight-line episodes 001 to 006 if nothing failed), checked and backed up, and go on the exclusion list so they never enter a dataset. We write the winner and the loser, with the numbers, into the method decision record in the documentation folder, together with how we define a step: the tip moving forward and sideways at a fixed height, as the AI Project Guide explains.

Manual work: six pushes, 30 to 45 minutes, both of us at the robot.

Step 5: the pilot and the replay

The pilot is five real pushes on the straight line with the winning method, under the next five free episode numbers, recorded, checked and backed up like any session.

Then the replay. A script drives the robot through one pilot push again from its table of tip positions, step by step under the speed cap (the top speed set in `config.py`, 20 % of the robot's maximum), on the same sheet with the puck back at the start, and nobody

touching the arm. A step already sent cannot be stopped from the keyboard; Ctrl+C only holds back the next one, so whoever is not running the script keeps a hand on the robot's stop button for the whole replay. If the puck reaches the goal within one puck radius, the recording contains everything a policy will need, and the way we define a step is confirmed. If it does not, we replay the same push from the joint angles already in its table; if that reaches the goal, the decision record says the step is defined in joint angles. Only if both replays miss do we record the pilot again.

The replay also gives the first real number for the robot's loop: the loop period, the time one replayed step takes from sending the move to its end, compared with the spacing in the recording. It goes into `config.py`.

Phase 2 is finished when the pilot session is in `03_data/raw` and on the second disk, its five pushes pass the session check, the clock test's spread is under one picture interval, one replay reached the goal, the six trial pushes are on the exclusion list, and the decision record names the method, the step definition and the loop period. The checklist at the end lists the same things to tick.

Manual work: five pushes and one replay watched, 30 to 45 minutes.

Who does what

At least half of each grade is individual, so the split is written down before we start, and every push, script and note carries the name of whoever made it. This is a starting proposal; swap as you like, but write the swap down.

Task	Owner	The other one
Workspace: measuring, taping, pusher, start pose, photos	Both, at the robot together	
Recording script (step 2), written with me	Emranul	Reviews and runs it on a test push
Session check, clock test and backup scripts (step 3)	Marshal	Reviews and runs them on the trial session
Pushes in the method trial and the pilot	Both: each records half, the other runs the script	
Safety during any motion: hand near the stop button, area clear	Whoever is not pushing	
Backup at the end of every session	Marshal	Emranul checks the copy exists

Task	Owner	The other one
Decision record and session notes	Emranul writes, Marshal checks	

Two rules keep this honest: nobody records alone, because one person pushes while the other runs the script and watches the arm; and `session_notes.md` names the person for every push, so the share of the work is visible in the data itself.

Mistakes to expect in this phase

What goes wrong	What you see	The fix
Learning mode switched on with nobody holding the arm	The arm drops onto the table	Hold the arm at the elbow before switching, every time; never switch it from across the room
The sheet moved between pushes	The tip's path drawings drift from push to push	All four corners taped, corner marks on the table, check before each push
A hand in the pictures	Fingers on the contact sheet	Hold at the elbow, or drive from the keyboard
The clock test fails	The three taps disagree by more than 110 ms	Close NiryoStudio and every other program using the robot or the camera; repeat the test; reseat the camera cable
Pushing too fast	Blurred pictures; the puck slides on after the pusher stops	Slow down; the script warns when the tip moved more than 2 cm between rows
Episode numbers restart	Two files with the same name	The script reads the highest number in raw before it starts; nothing is ever renamed by hand
The pusher touches the puck too high	The puck tips over	Lower the pushing height; the contact must be below half the puck's height

What goes wrong	What you see	The fix
The camera focus got touched	Sharpness differs from the reference picture	Compare with the reference at the start of each session using `view_camera.py`; do not turn the ring
The cable link drops mid-push	The script loses the robot	Check the Ethernet cable at both ends and that the robot's address in `config.py` answers to ping; discard the push, reconnect, continue
The backup is forgotten	A session exists on one disk only	The backup script is the last step of every session, and the checklist asks for it

Checklist before we call phase 2 finished

- [ ] Puck diameter, height and weight written into `config.py`
- [ ] Straight-line sheet taped, corners marked, line length noted
- [ ] Pusher fitted; it touches the puck below half its height; the camera sees the puck and the line ahead
- [ ] Pushing height, start pose and workspace box in `config.py`; the start script moves the arm there and back
- [ ] Set-up photographed with a tape measure in the frame
- [ ] Recording script records a test push and discards it; nothing from testing is in raw
- [ ] Clock test runs and writes `sync.json`; spread under one picture interval
- [ ] Session check runs on a session and produces the plots and the contact sheet
- [ ] Backup script copies a session to the second disk and verifies it
- [ ] Method trial done: six pushes recorded, scored, on the exclusion list; the winner in the decision record
- [ ] Pilot: five pushes recorded, checked, backed up
- [ ] One pilot push replayed; the puck reached the goal within one puck radius
- [ ] Loop period and replay timing written into `config.py`
- [ ] Decision record names the method, the step definition and the loop period